

Docker

A Beginner's Guide: Installation to Running Your Django + PostgreSQL Project

This guide walks through Docker from scratch: what it is, how to install and verify it, how to containerize a Python/Django project that depends on a specific Python version and PostgreSQL, and a command reference for everyday use. Follow the sections in order the first time.

1. What is Docker? (Core Concepts)
2. Installing Docker on Ubuntu
3. Verifying the Installation
4. Core Docker Commands (Cheat Sheet)
5. Writing a Dockerfile for a Python/Django App
6. Writing docker-compose.yml (App + PostgreSQL)
7. Full Project Setup: Step by Step
8. Everyday Commands Reference
9. Beginner Learning Path (What to Practice Next)

1. What is Docker? (Core Concepts)

Docker packages an application together with everything it needs to run — the exact programming language version, libraries, and system tools — into a single unit called an **image**. That way the app runs identically on any machine, regardless of what is or isn't installed on it.

Key terms

Term	What it means
Image	A read-only blueprint: your app's code + dependencies + OS bits. Like a recipe.
Container	A running instance of an image. Like the dish actually cooked from the recipe.
Dockerfile	A text file with instructions for building an image (what to install, what to copy, what to run).
docker-compose	A tool to run multiple containers together (e.g. your app + a database) with one command.
Volume	A way to persist or share files between your computer and a container.
Port mapping	Connects a container's internal port to a port on your machine, so you can open it in a browser.

Why this matters for your case: your project needs an exact Python version and specific package versions. Instead of changing your real machine's Python installation or fighting with pip, Docker lets you define the exact environment your project wants, and it runs the same way every time.

2. Installing Docker on Ubuntu

Note: if `docker --version` already works on your machine, Docker is installed — skip to Section 3.

Step 1 — Remove any old/partial Docker packages (optional, only if previous attempts failed)

```
sudo apt remove docker docker-engine docker.io containerd runc
```

Step 2 — Install prerequisites

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release
```

Step 3 — Add Docker's official GPG key and repository

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo \"${VERSION_CODENAME}\") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Step 4 — Install Docker Engine

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Step 5 — Start Docker and enable it on boot

```
sudo systemctl enable --now docker
```

Step 6 — Let your user run Docker without sudo

Important: log out and log back in (or reboot) after this step. Group membership only applies to new login sessions.

3. Verifying the Installation

```
docker --version  
docker compose version  
docker run hello-world
```

If the last command prints "Hello from Docker!", your installation is fully working — Docker downloaded a tiny test image, ran it in a container, and it printed a message back to you.

If you get a "permission denied" error instead, it means the usermod/group step (Section 2, Step 6) hasn't taken effect yet — log out and back in, then try again.

4. Core Docker Commands (Cheat Sheet)

Command	Purpose
<code>docker build -t name .</code>	Build an image from a Dockerfile in the current folder
<code>docker images</code>	List all images on your machine
<code>docker rmi <image></code>	Delete an image
<code>docker run <image></code>	Start a new container from an image
<code>docker run -it <image> bash</code>	Start a container and open an interactive shell inside it
<code>docker run -d <image></code>	Run a container in the background (detached)
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers, including stopped ones
<code>docker stop <id></code>	Stop a running container
<code>docker start <id></code>	Start a previously stopped container
<code>docker rm <id></code>	Delete a stopped container
<code>docker logs <id></code>	View a container's output/error logs
<code>docker exec -it <id> bash</code>	Open a shell inside an already-running container
<code>docker compose up</code>	Start all services defined in docker-compose.yml
<code>docker compose up --build</code>	Rebuild images, then start all services
<code>docker compose up -d</code>	Start all services in the background
<code>docker compose down</code>	Stop and remove all services from compose
<code>docker compose logs -f</code>	Follow logs from all compose services live
<code>docker compose exec web bash</code>	Open a shell inside the 'web' service container

5. Writing a Dockerfile for a Python/Django App

A Dockerfile is a plain text file (named exactly **Dockerfile**, no extension) placed in your project's root folder. Each line is an instruction for building your image.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

# Install everything possible; continue even if one bad pin fails
RUN pip install --no-cache-dir -r requirements.txt || true

# Patch any pins that don't exist on PyPI (adjust as needed)
RUN pip install --no-cache-dir asgiref psycopg2-binary

COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

What each line does

Line	Meaning
FROM python:3.11-slim	Start from an official image that already has Python 3.11 installed
WORKDIR /app	All following commands run inside /app in the container
COPY requirements.txt .	Copy just this file first, so Docker can cache the install step
RUN pip install ...	Executes during the image build (not when the container runs)
COPY . .	Copy the rest of your project files into the image
EXPOSE 8000	Documents which port the app listens on
CMD [...]	The command that runs when a container starts from this image

.dockerignore (place next to the Dockerfile)

```
menv/
__pycache__/
*.pyc
.git/
.env
```

This keeps your virtual environment and junk files out of the image, making builds faster.

6. Writing docker-compose.yml (App + PostgreSQL)

You do not need to install PostgreSQL on your machine. docker-compose runs PostgreSQL in its own container, alongside your app's container, and connects them automatically.

```
services:
  db:
    image: postgres:15
    restart: always
    environment:
      POSTGRES_DB: library_db
      POSTGRES_USER: library_user
      POSTGRES_PASSWORD: library_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./app
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      DB_NAME: library_db
      DB_USER: library_user
      DB_PASSWORD: library_pass
      DB_HOST: db
      DB_PORT: 5432

volumes:
  postgres_data:
```

Update settings.py to read these environment variables

```
import os

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME'),
        'USER': os.environ.get('DB_USER'),
        'PASSWORD': os.environ.get('DB_PASSWORD'),
        'HOST': os.environ.get('DB_HOST'),
        'PORT': os.environ.get('DB_PORT'),
    }
}
```

Note: this edit is in settings.py, not requirements.txt — if settings.py is also off-limits, say so and the values can instead be supplied purely through docker-compose without editing code.

7. Full Project Setup: Step by Step

Step 1: Confirm Docker works

```
docker run hello-world
```

Step 2: Check your Django/Python requirements

```
grep -i django requirements.txt
```

```
grep -i psycopg requirements.txt
```

Step 3: Create Dockerfile and .dockerignore

(see Section 5 – place both in project root)

Step 4: Create docker-compose.yml

(see Section 6 – place in project root)

Step 5: Update settings.py DATABASES block

(see Section 6)

Step 6: Build and start everything

```
docker compose up --build
```

Step 7: Run database migrations (separate terminal, first time only)

```
docker compose exec web python manage.py migrate
```

Step 8: Create a Django superuser (optional)

```
docker compose exec web python manage.py createsuperuser
```

Step 9: Open the app

```
http://localhost:8000
```

Step 10: Stop everything when done

```
docker compose down
```


8. Everyday Commands Reference

Once things are set up, these are the commands you'll use day to day:

Task	Command
Start the whole project	<code>docker compose up</code>
Start in background	<code>docker compose up -d</code>
Rebuild after Dockerfile changes	<code>docker compose up --build</code>
Stop the project	<code>docker compose down</code>
View logs of all services	<code>docker compose logs -f</code>
Run a Django management command	<code>docker compose exec web python manage.py <command></code>
Open a shell inside the app container	<code>docker compose exec web bash</code>
Open a psql shell inside the DB container	<code>docker compose exec db psql -U library_user -d library_db</code>
See what's currently running	<code>docker ps</code>
Free up disk space (remove unused images)	<code>docker system prune</code>

9. Beginner Learning Path (What to Practice Next)

1. Mental model

An image is a recipe; a container is the dish cooked from it. Re-read Section 1 until this feels natural.

2. Play in a disposable container

Run: `docker run -it ubuntu bash` — you get a free-to-break Ubuntu shell. Try `apt install`, break things, then just type `exit`. Nothing on your real machine is touched.

3. Read your own Dockerfile line by line

Go back to Section 5 and match each instruction (`FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`) to what it actually does.

4. Understand `-p` and `-v`

`-p 8000:8000` exposes a container's port to your machine. `-v $(pwd):/app` syncs your folder live into the container so code edits show up without rebuilding.

5. Learn docker-compose properly

You're already using it. Try adding a third service (e.g. `redis`) to `docker-compose.yml` as practice.

6. Explore image layers

Run `docker history` to see how an image is built up in layers — helps you understand why builds get cached and fast.

7. Official interactive tutorial

docs.docker.com/get-started — hands-on, free, no signup needed to read.

8. Eventually: multi-stage builds

Once comfortable, look into multi-stage Dockerfiles to make production images smaller and cleaner.

Tip: keep this PDF next to your project. Sections 5–7 are the ones you'll return to most while getting the library management system running for the first time.